



European
Power Platform
Conference

WELCOME
TO

COPEN
HAGEN 2 0
2 6



JOIN THE
CONVERSATION
#EPPC26



Developer ALM Lab

Source Control, Build System & CI/CD

Tomas Prokop

Solution Architect @ NETWORG

 [prokopus.com](https://www.prokopus.com)



Developer ALM Lab

Source Control, Build System & CI/CD

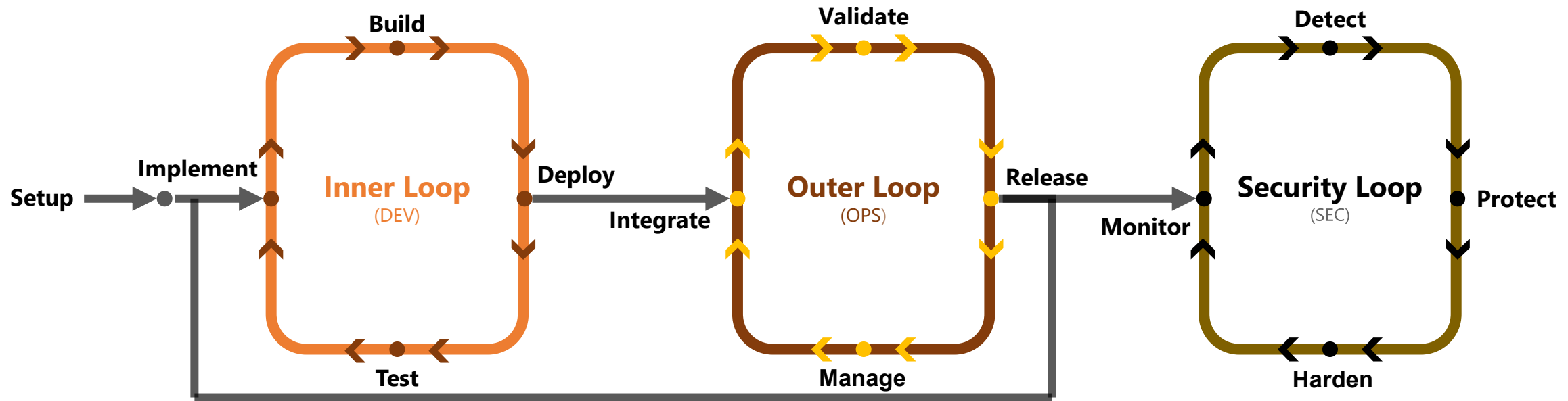
Matej Samler

Senior Staff Engineer @ NETWORG

 /metju-samler

Overview

Automation of Dev Loops



What You'll Build

Overview

Over the day, you build a real Power Platform app and the full ALM around it:

- **A Power Platform app, as a monorepo**
Apps, data model, plugins, PCF controls, solution components
- **Git as the single source of truth**
Environments become disposable, rebuilt from the repo.
- **A CI/CD pipeline on GitHub Actions**
Build gates on every PR, deploy via FIC/OIDC.
- **Infrastructure as code**
Utilize infrastructure as code for provisioning that also works as a living documentation
- **An AI agent doing the work**
You direct it, with a manual fallback at every step

Why This Lab Exists

Overview



Generating code is cheap now

Keeping it working as it changes is the hard part.

This lab is the discipline for it

Source control, a build, tests, releases. Standard software practice for 20 years, now agent-paced.

Power Platform in the Agentic Era

AI

- As agents write more of the code, the code gets cheap and disposable
- The foundation under it gets more valuable



The system of record stays

Power Platform and Dataverse hold your data model, business rules, permissions, and audit trail. Agents transact against it.

A prompt asks an agent to follow a rule. A system of record/action enforces it.

Where We Think This Is Heading

Our bet on the direction, not a settled fact

AI

It converges on one model:

- **Everything lives in one git repo**
Code, config, tests, the pipeline.
- **Agents work that repo directly**
Full file system access: they run scripts and read every file.

No One Size Fits All

ALM

There's no single right way to run ALM on Power Platform.
Teams pick a level that fits how much control and automation they need.

Manual
export / import

Platform-native
ALM tools

Custom, code-first
pipeline

simple cases

full automation

easier onboarding

more effort

i Every level here is a valid choice depending on scenario and team

You Can Already Do That

ALM

Agents already write real code on Power Platform:

- **PCF controls and code apps**
TypeScript and React, already AI-generated. Plugins and backend in C# are next.
- **For your code, the platform's ALM stops at deployment**
It packages and ships the components.

Version control, configuration, code review and automated tests, you bring those yourself.

Multiple ALM Options

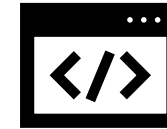
ALM



Platform-native ALM

Solution import/export and Power Platform Pipelines.

The simple version.



Code-first ALM

(hands-on, most of the day)

Your own repo, synced to Power Platform from the terminal.

Source control, builds, tests, releases, change management.

Why This Matters

ALM

ALM in Power Platform has direct cost and scalability consequences

- ✅ **Cheap and fast at first** - One person modifying one solution works for small scale
- ⚠️ **Scalable** - Ad-hoc edits and manual imports become expensive as teams grow
- ❌ **Unacceptable** - "My solution import takes 40 minutes only to fail at the end"

Git vs. a Source Control Management Server

Source
Control

Git is a distributed version-control system, the whole repo and its history live on **your machine**. GitHub and Azure DevOps host the **shared remote** and add what Git has no concept of: **pull requests, CI/CD, issue tracking, access control**.



Azure DevOps

Microsoft's all-in-one toolbox.
Code hosting, work-item boards,
build pipelines, package feeds.

**Mature, enterprise-grade, lots of
configuration.**



GitHub

The world's code-sharing hub, now a
platform. Code hosting, issues &
projects, Actions, package feeds.

**AI-native, Copilot built in, biggest
ecosystem.**

Open to Edit, or Locked Down

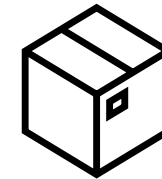
Transfer code metadata between environments, modularization, layering and customization.

Solution Framework



Unmanaged

References to individual components
Everything can be modified
For development purposes only
Deletion does not remove the components
Export as Managed and Unmanaged



Managed

Locked for customizations
Components cannot be added or removed
No export possible
Recommended for test, UAT, SIT and PROD environments



The Pipeline (the build gate)

CI/CD

What stops a broken change from reaching `main`?

All day the agent commits and opens PRs. The only check is a human reading the diff, which doesn't scale and fails when the agent works unattended.

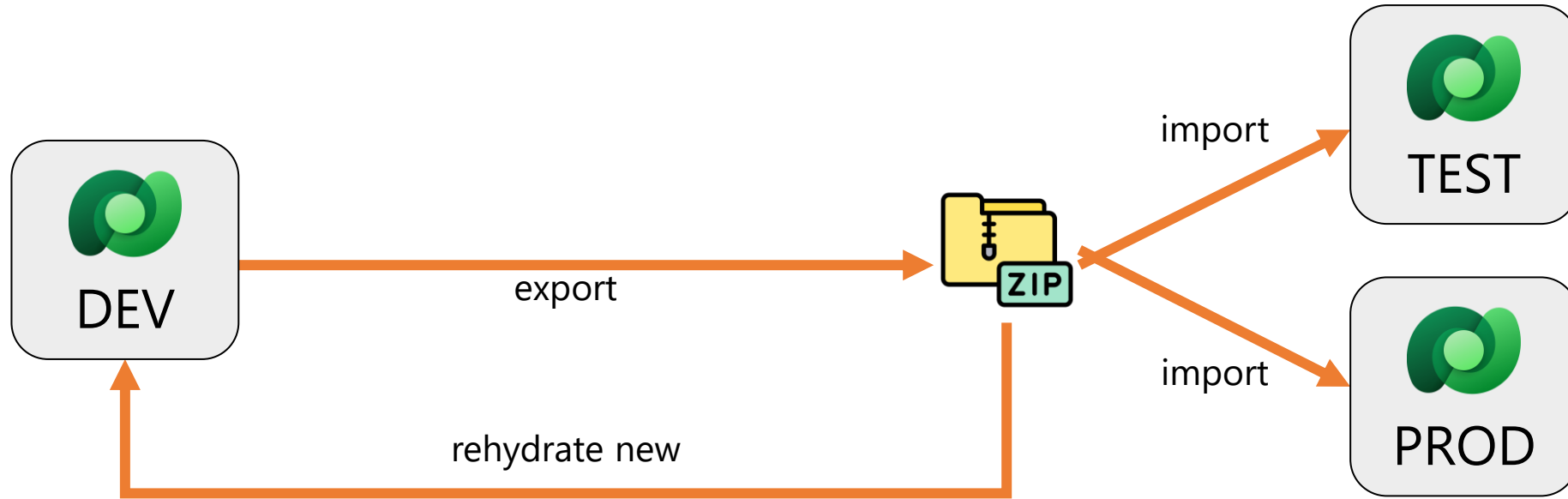
Answer: **an automatic gate, a pipeline that runs on every push.**

You push code → Build → Test → Deploy

Triggered by your push, no manual commands. A failed step turns the status check red, and branch protection blocks the merge.

Without CI/CD

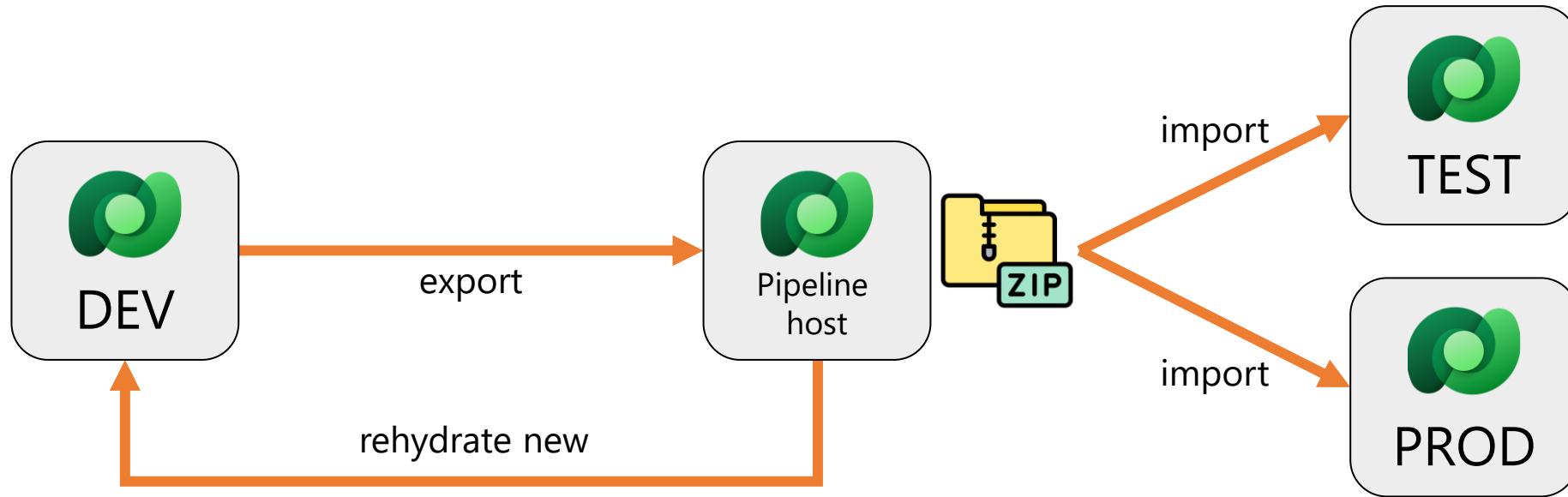
ALM



The whole process and configuration has to be done manually.

Power Platform Pipelines

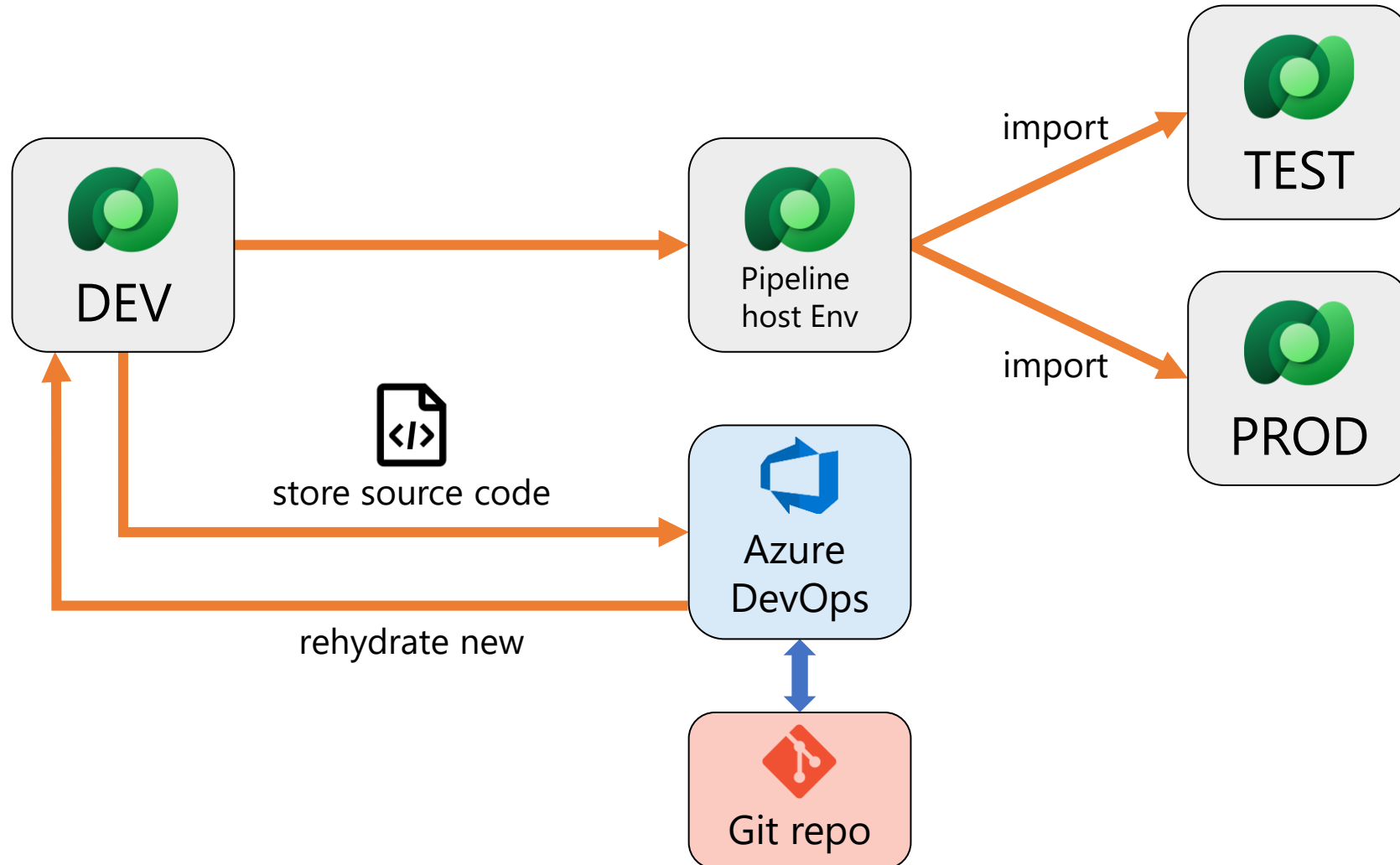
ALM



Pipelines automate solution deployments between Power Platform environments.

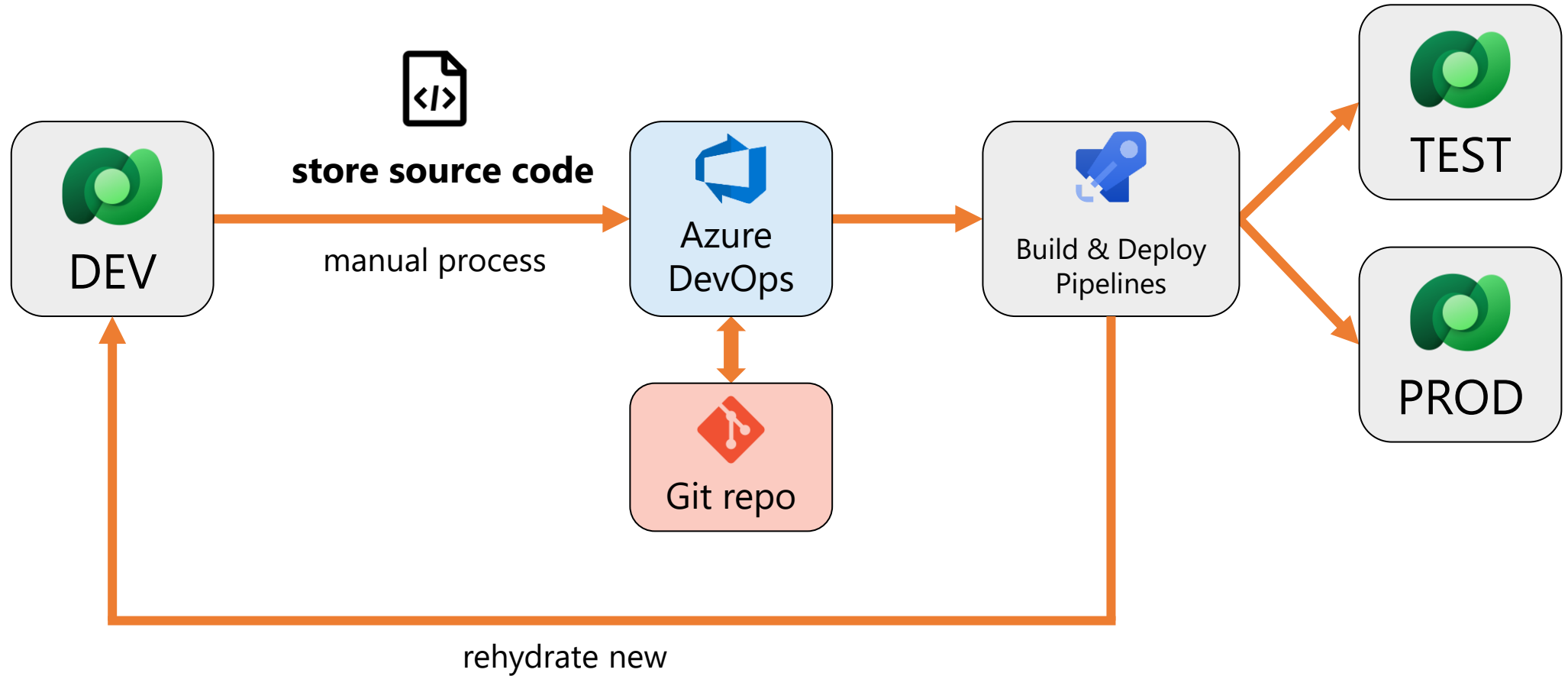
Git integration + Power Platform Pipelines

ALM



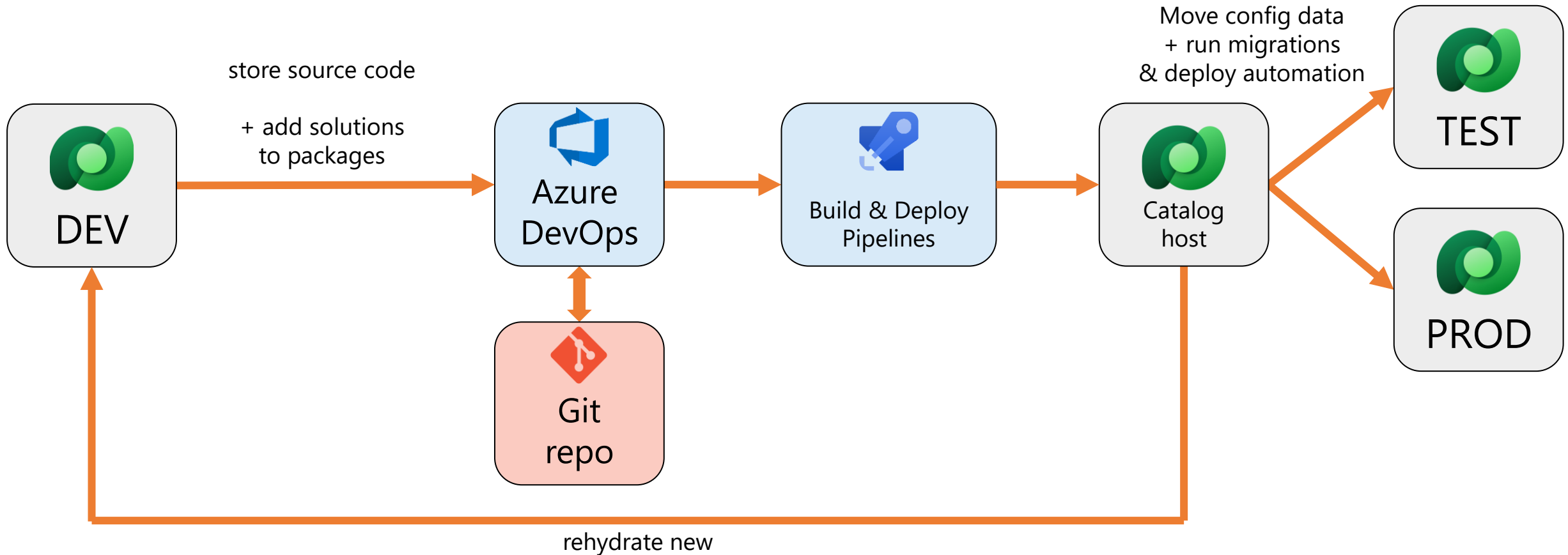
Manual Git + Azure DevOps Pipelines + Power Platform Build Tasks

ALM



Manual Git + Package Deployer + Azure DevOps Pipelines + Power Platform Build Tasks + Power Platform Catalog

ALM



What Happens When AI Writes Tests

Testing

Shipping 10 PRs a day with coding agents. Hundreds of unit tests in the codebase. Spot the problem.

"AI doesn't respect why we put things in place. It just fixes the tests instead."

Code works but isn't well designed. Cross-cutting PRs are hard to review.

The agent changes things it doesn't understand, and there's no architect in the loop.

1 Simplification

The agent doesn't test what it's supposed to.

It writes a thinner test that's easier to pass, and ships green. The intent of the original test is lost.

2 Test corruption

The agent edits an existing test to make it pass.

Now the test silently validates the new (possibly broken) behavior. The guard is gone, and no one notices in code review.

This is a new problem. AI is both the best tool for generating tests *and* inherently motivated to make them pass.

The Fundamental Conflict

Two forces pulling in opposite directions.

Testing

AI is non-deterministic

Same prompt

different code each time

Inference cost per run

(time + dollars)

Drifts with model versions

and provider updates

Tests must be deterministic

Same input

same result, every run

Pipelines run hundreds of times a day

(must be cheap)

Must reproduce months later

with the same outcome

vs

The resolution:

Use AI where it shines (authoring tests from behavior), keep it out where it hurts (executing them in CI).



.NET CLI

Developing, building,
running and publishing
.NET applications



dotnet new
Creates new .NET
projects



dotnet build
Compiles .NET
projects



dotnet test
Runs tests using a
test runner



dotnet publish
Packs apps for
deployment

aka.ms/dotnet-cli-docs



PAC CLI

Managing
and automating tasks within
Power Platform



pac solution
Manages
solutions



pac package
Packages Power
Platform components



pac data
Handles data
operations



pac admin
Administers Power
Platform

aka.ms/PowerPlatformCLI



Azure CLI

Simplifies and automates
operations
of Azure services



az devops
Manages Azure
DevOps services



az pipelines
Creates and manages
Azure DevOps
pipelines



az repos
Manages
repositories



az ad app
Manages Entra ID
app registrations

aka.ms/azcli-docs



Terraform CLI

Provisions and manages
infrastructure as code



terraform init

Sets up the working
directory



terraform plan

Previews infrastructure
changes



terraform apply

Applies infrastructure
changes



terraform destroy

Tears down
infrastructure

developer.hashicorp.com/terraform/cli



TXC CLI

Code-first toolkit for Power Platform and Dataverse development



txc workspace

Scaffolds components locally



txc env

Syncs local source with an environment



txc data

Query & modify Dataverse records



txc config

Manages identity & profiles

talxis.com/cli

- Power Platform's code-first commands are split across pac CLI, Dataverse CLI, Code Apps CLI, PP skills, Dataverse Skills, Python DV SDK, ... and neither covers the scenario fully.
- **txc** is an unofficial, open-source CLI that does: scaffold components locally, sync them with an environment, validate the source code, and work with data.
- We use it in the lab to save time. Every step also works with the Microsoft tools, just with more friction, so **txc** is recommended here, not required.

DevKit Build



Power Platform MSBuild Tasks

Extensions for .NET projects targeting Power Platform which let you customize the build



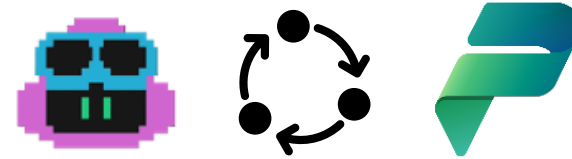
Schema validation and version numbers



PAC CLI integration WIP (issue #2)

talxis.com/sdk

The Lab



Agentic Code-first Power Platform ALM

The full path from an empty repo to a deployed, tested Power Platform app.



One source of truth



Customizations as reviewable diffs

talxis.com/lab

Disclaimer: All of the above are work-in-progress & serve as examples

Interested in the full workshop? Get in touch.

The full ALM lab: source control, the solution framework, CI/CD, testing, and more.

a) Sign up for the DevKit

The open-source tooling the lab runs on. You get the updates and the materials.



<https://talxis.com/devkit-subscribe>

b) Contact us on LinkedIn



Tomas Prokop

Solution Architect
@ NETWORG

 [prokopus.com](https://www.linkedin.com/company/prokopus.com)



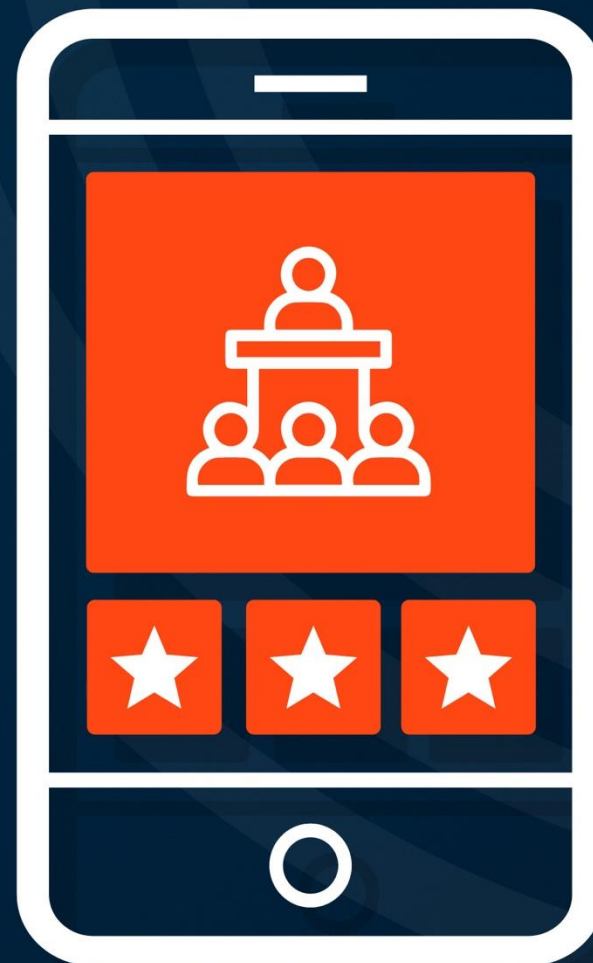
Matej Samler

Senior Staff Engineer
@ NETWORG

 [/metju-samler](https://www.linkedin.com/company/metju-samler)



Please rate
this session
on the app



cvent

